Cetus-DLMM

Audit Report

MOVEBIT

contact@bitslab.xyz

https://twitter.com/movebit_

Thu Sep 25 2025

Cetus-DLMM Audit Report

1 Executive Summary

1.1 Project Information

Description	Cetus is a pioneer DEX and liquidity protocol built on the Sui blockchain.
Type	Dex
Auditors	poetyellow, Bear Two, jolyon
Timeline	Mon Jul 21 2025 - Thu Sep 25 2025
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/CetusProtocol/cetus-dlmm
Commits	cd6b4e90ed6648a73bf7af64326caccced54bb29 996091a88c9b5a85fb78681d2367919a0e8646f1 f51c7e16a44fdf0ecc7441fc29b247f1d6713a90 c0cb989e1369952b7064adffa3cffe549fc42668 9ffb12f7960de6b610482c8f7291c6299b3b8c9d c318ee1050ecfb6d604182fe613c4bed78b4c328 160846ec4eb36bd5b9db416c015069a8c7d7e759 efdb6375f1e3192e30f2c3ac705929bfb1bfa8ee f442ca0db0bd8c0e502029831fe3a1a2e960d038 80530ae53d162e4da26aeb665de305e17fb7768e

	3ad93678421b8bad01e7eca2cb4eda4b02c83503 949ca8154c8ebf73d00d05f831b2947060542de4 abf32f00e8ff039833f0a4da9110d75ce21c05aa 16eabf284b0dd45d90c83d86cee2ea19c4cc4185

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
RES	packages/cetus_dlmm/sources/restriction.move	f61d90d99938ce330ecc0d4b7a9c661a0ff9a6cd
ACL	packages/cetus_dlmm/sources/acl.move	0cf32dab476140a04a0e0745fcbbaac1c3917c63
ACA	packages/cetus_dlmm/sources/admin_cap.move	38d5ee405bcfb880a2debef7b7c7b50d466e02bf
PAR	packages/cetus_dlmm/sources/parameters.move	9e38f40004af309f8823e6cdb601f64d69c9b3a7
POS	packages/cetus_dlmm/sources/position.move	1c26687a2b324a5b88b79f56e6a2d8b09fae7e59
VER	packages/cetus_dlmm/sources/versioned.move	ad9f8d27ad4bd4f019813122badb4cd0d0c389d3
PMA	packages/cetus_dlmm/sources/price_math.move	91ea8d7642532694fb87d9e2e9421e007a1a8994
DMA	packages/cetus_dlmm/sources/dlmm_math.move	8c07a770679af115dfa6c1d527b470a55a153c57
CON	packages/cetus_dlmm/sources/constants.move	8f8c69dc99bef47629833f8a944069907230cb0b
PAR1	packages/cetus_dlmm/sources/partner.move	0dc4ac42da7991da97ed8c7ab9f6b5560f0c42d0

CON1	packages/cetus_dlmm/sources/config.move	f3d83c3f1a51360bae02e8473794a6222e5b7109
REW	packages/cetus_dlmm/sources/reward.move	a68cd8475cc3017eae183e38634c3b9e578f1b32
POO	packages/cetus_dlmm/sources/pool.move	d1c26de6cb0e3b9c7f11be5a80d7930899a69888
PMO	packages/cetus_dlmm/sources/pool_mode.move	b4f3fd8b7e92598345ee41a56963730003855d0e
REG	packages/cetus_dlmm/sources/registry.move	c5f93fa043a113239e8263faf7b16a64b30d28bb
BIN	packages/cetus_dlmm/sources/bin.move	a9808a4a8f13feb7b4829b7c51db9201b60efc7e
ACA	packages/cetus_dlmm/sources/admin_cap.move	359bd5b6f03e7d86692cc727e67880b1fa5e9e9f
POS	packages/cetus_dlmm/sources/position.move	1c0297f54488b536bc64f53fef841f596309f561
VER	packages/cetus_dlmm/sources/versioned.move	89877a0202401ea72fef82fa28f5f5e7f3afeb0e
DMA	packages/cetus_dlmm/sources/dlmm_math.move	975480abaa81b63fb5791663796ec702e8ac4a64
CON	packages/cetus_dlmm/sources/constants.move	39346453ba110c71f3613c7390bf1254123f60e8
CON1	packages/cetus_dlmm/sources/config.move	44c71ece2da0ab70e324fb40d995c2ee1a981905

REW	packages/cetus_dlmm/sources/reward.move	2862f7709291eb62f37b25ab07fb981d86cd902a
POO	packages/cetus_dlmm/sources/pool.move	64b3d0b615f6350ea1ec8f4d273bc327047e95d6
REG	packages/cetus_dlmm/sources/registry.move	ca23689e85bf0a1e87c874b998668dab007df0da
BIN	packages/cetus_dlmm/sources/bin.move	84f8c187dc6a738dcd6bf6c2a2fd46c2115368da
SWA	packages/dlmm_router/sources/swap.move	e120b49198e9460cd12a7f4631dbfbbfa6b3ec05
ALI	packages/dlmm_router/sources/add_liquidity.move	5f37ff3a9895df9a72f60695b5c88a4a05429f75
RES	packages/cetus_dlmm/sources/restriction.move	63051e317833c2cf25fe379d35a45410dc9a2d9b
ACA	packages/cetus_dlmm/sources/admin_cap.move	f3df7499319829e61423f95c06c871e78af6994a
VER	packages/cetus_dlmm/sources/versioned.move	e51366d6a350129b27675ab46b633afc505cf558
PMA	packages/cetus_dlmm/sources/price_math.move	f1d7d7a0a89a7e2f7b6d0f3d713bf85011050086
DMA	packages/cetus_dlmm/sources/dlmm_math.move	aef5b262d01b77b2512cbaa25a5e1f5967556919
CON	packages/cetus_dlmm/sources/constants.move	35f9f055ccf8ee4fc06692607aa45bcc7142551f

PAR1	packages/cetus_dlmm/sources/partner.move	62492adb1a08868f1368687eb221f7978d633e2d
CUR	packages/dlmm_router/sources/curve.move	ec4c639e6030ced95d3356883e79971399efacf5
SPO	packages/dlmm_router/sources/spot.move	444e3c336266cdc5f5009031245d1e032d666787
BAS	packages/dlmm_router/sources/bid_ask.move	31f3e7a2ab63fe3d58d96699e082e28b1a8e818d
UTI	packages/dlmm_router/sources/utils.move	a67542f0add4807ddc0814b281a1239e1e21780c
PWR	packages/dlmm_router/sources/position_wrapper.move	b8f3d4542c64826f5d96b54aaca489e9bf0c892e
ACL	packages/cetus_dlmm/sources/acl.move	6cbff52003075b581ede1517d4c3125ebbd53789
PAR	packages/cetus_dlmm/sources/parameters.move	4cdcc603a17a84fc24834e91f81ea885ce4710be
POS	packages/cetus_dlmm/sources/position.move	0fb55208d3efbb5debf38b814f9c744c6c77088f
CON1	packages/cetus_dlmm/sources/config.move	fd5f0dba2515c5bc666a62f3eb0539a8b62102db
REW	packages/cetus_dlmm/sources/reward.move	17209fa857ffa5e530dcc6942aacbfb4a262488f
POO	packages/cetus_dlmm/sources/pool.move	064d81c2fc7b54f1351971705c1e54c47e11ac84

REG	packages/cetus_dlmm/sources/registry.move	3adc968af1b2412c87ad57065cfa938c3498b3c8
BIN	packages/cetus_dlmm/sources/bin.move	49fca05364c39a65fa9aa66c77bc115f7916cac2

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	23	23	0
Informational	4	4	0
Minor	9	9	0
Medium	8	8	0
Major	1	1	0
Critical	1	1	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** and **"Formal Verification"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by Cetus to identify any potential issues and vulnerabilities in the source code of the Cetus-DLMM smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 23 issues of varying severity, listed below.

ID	Title	Severity	Status
BIN-1	Incorrect Fee Allocation Logic in Swap Operations Leads to Duplicate Protocol Fee Collection	Medium	Fixed
CON-1	Unprotected Public Function Has Mutable Access to Shared State	Minor	Fixed
DMA-1	Variable Naming Errors in Code	Minor	Fixed
PMA-1	`price_math::pow` Does Not Handle Base = 1 as a Special Case	Minor	Fixed
PMA-2	Missing Error Constants in Assert Statements	Informational	Fixed
POO-1	Fake Openings Lead to Malicious Withdrawal of Pool Fees and Funds	Critical	Fixed
POO-2	Incorrect Permission Check in `collect_position_reward`	Major	Fixed

POO-3	Missing Position-Pool Matching Validation in `add_liquidity` and `remove_liquidity` Functions	Medium	Fixed
POO-4	Ref Fee Calculation Based on Protocol Fee Leads to Double Charging	Medium	Fixed
POO-5	Dead Position Risk	Medium	Fixed
POO-6	Position Boundary Check Bypass Vulnerability	Medium	Fixed
POO-7	Users are Not Paying Protocol Fees	Medium	Fixed
POO-8	Missing Validation for Length Consistency of `bins` `amounts_a` and `amounts_b` in `add_liquidity`	Minor	Fixed
POO-9	Event Log and State Inconsistency May Mislead Users	Minor	Fixed
POS-1	Typographical Error in Variable Name in `init` Function	Informational	Fixed
POS-2	Misleading Error Code Used in `restore_position_info` Function	Informational	Fixed
REG-1	Missing Token Pair Sorting in Pool Key Generation	Minor	Fixed
REG-2	Ambiguous Boundary Definition for active_id	Minor	Fixed
REG-3	Missing Version Check	Informational	Fixed

REW-1	Missing Reward Release Update When Breaking from Settlement Loop	Medium	Fixed
REW-2	Accumulated but Unwithdrawable Refunded Rewards in `reward_refunded`	Medium	Fixed
REW-3	Inflating `reward_released` Statistics	Minor	Fixed
PAR1-1	Missing Validation of Current Time vs End Time in `create_partner`	Minor	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the Cetus-DLMM Smart Contract :

Admin

- `update_permissions` - Update pool operation permission settings
- `emergency_pause` - Emergency pause reward distribution
- `mint_protocol_fee_collect_cap` - Mint protocol fee collection capability for specified address
- `collect_protocol_fee` - Collect accumulated protocol fees from pool
- `collect_protocol_fee_with_cap` - Collect protocol fees using capability
- `make_reward_public` - Make reward manager public
- `make_reward_private` - Make reward manager private
- `initialize_reward` - Initialize new reward token type
- `add_reward` - Add reward tokens to pool

User

- `flash_swap` - Execute flash swap operation
- `repay_flash_swap` - Repay flash swap
- `flash_swap_with_partner` - Execute flash swap with partner referral fees
- `repay_flash_swap_with_partner` - Repay flash swap with partner referral fees

Liquidity Provider

- `open_position` - Open new liquidity position
- `close_position` - Close position and return all tokens and fees
- `destroy_close_position_cert` - Destroy close position certificate

- `add_liquidity` - Add liquidity to existing position
- `repay_add_liquidity` - Repay tokens for liquidity addition
- `remove_liquidity` - Remove liquidity from position
- `update_position_fee_and_rewards` - Update position fee and reward tracking
- `collect_position_fee` - Collect accumulated fees from position
- `collect_position_reward` - Collect accumulated rewards of specified type from position
- `take_reward_from_close_position_cert` - Extract specified reward type from close position certificate

4 Findings

BIN-1 Incorrect Fee Allocation Logic in Swap Operations Leads to Duplicate Protocol Fee Collection

Severity: Medium

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/bin.move#327-331

Descriptions:

A critical fee allocation error has been identified in the swap operation's fee handling logic. The issue arises from inconsistent fee processing between the bin level and the pool level: In the swap function within `bin.move`:

```
bin.amount_a = bin.amount_a + amount_in - fee_amount;
bin.amount_b = bin.amount_b - amount_out;
bin.accumulate_fee(fee_amount, 0); // Allocates the entire fee to liquidity providers
```

In the swap function within `pool.move`:

```
protocol_fee = protocol_fee + calculate_fee_inclusive(fee, protocol_fee_rate);
// ...
if (a2b) {
    pool.protocol_fee_a = pool.protocol_fee_a + protocol_fee;
} else {
    pool.protocol_fee_b = pool.protocol_fee_b + protocol_fee;
};
```

Bin Level: accumulate_fee(fee_amount, 0) allocates the entire swap fee to liquidity providers.

Pool Level: Simultaneously, the protocol fee is calculated and collected from the same fee.

Result: The protocol fee is effectively double-charged, meaning an additional fee is levied on users.

Suggestion:

Modify the bin-level swap function to ensure the protocol fee is correctly deducted:

```
// In the swap_exact_amount_in function
let protocol_fee_amount = calculate_fee_inclusive(fee_amount, protocol_fee_rate);
let lp_fee_amount = fee_amount - protocol_fee_amount;

// Update bin state
bin.amount_a = bin.amount_a + amount_in - fee_amount;
bin.amount_b = bin.amount_b - amount_out;
bin.accumulate_fee(lp_fee_amount, 0); // Only allocate the LP's entitled fee to liquidity providers

// Return protocol fee information for upper-level processing
(amount_in, amount_out, fee_amount, protocol_fee_amount)
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CON-1 Unprotected Public Function Has Mutable Access to Shared State

Severity: Minor

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/config.move#1363

Descriptions:

The public function `is_allowed_coin` accepts a mutable reference (`&mut GlobalConfig`) to the shared object `GlobalConfig` . Although the current implementation is read-only, its function signature is misleading and violates the principle of least privilege.

Suggestion:

Change the function signature to accept an immutable reference:

`public fun is_allowed_coin<Coin>(config: &GlobalConfig, ...): bool`

Resolution:

This issue has been fixed. The client has adopted our suggestions.

DMA-1 Variable Naming Errors in Code

Severity: Minor

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/dlmm_math.move#117;

packages/cetus_dlmm/sources/bin.move#258

Descriptions:

- `frist_score_for_swap` should be `first_score_for_swap`
- `detal_liquidity` should be `delta_liquidity`

Suggestion:

Fix the variable naming errors by correcting the spelling.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

PMA-1 `price_math::pow` Does Not Handle Base = 1 as a Special Case

Severity: Minor

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/price_math.move#109

Descriptions:

In the current implementation of `price_math::pow`, when the base is exactly 1.0 (represented as 1 << 64 in Q64.64 fixed-point format), the result is not guaranteed to be exactly 1.0 due to precision loss in the computation.

```
#[test]
fun test_pow() {
    use cetusdlmm::price_math::pow;
    let one: u128 = 1u128 << 64;

    let res = pow(one, i32::from(1));

    assert_eq!(one, res);

}
```

```
Running Move unit tests
[debug] "Assertion failed:"
[debug] 18446744073709551616
[debug] "!="
[debug] 18446744073709551617
[ FAIL    ] cetusdlmm::parameter_tests::test_pow
```

Suggestion:

Add explicit handling for the special case where the base is equal to 1.0, so that pow(1.0, x) always returns exactly 1.0 for any exponent.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

PMA-2 Missing Error Constants in Assert Statements

Severity: Informational

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/price_math.move#43,180

Descriptions:

Two assert statements in `sources/price_math.move` use hardcoded numeric error codes instead of descriptive constants:

```
assert!(exp < MAX_EXPONENTIAL, 1);
assert!(result != 0, 2);
```

This is inconsistent with the codebase pattern where other modules use descriptive error constants like EInvalidActiveId, EBinNotExists.

Suggestion:

Define descriptive error constants and replace numeric codes.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-1 Fake Openings Lead to Malicious Withdrawal of Pool Fees and Funds

Severity: Critical

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/pool.move#1033,1196

Descriptions:

1. `new_open_position_cert` calculates the active bin's fees and protocol fees based on the provided `active_id_amount_a` and `active_id_amount_b`, and directly adds them to the bin's fee growth global (see `apply_active_bin_composition_fees`).
2.However, at this point, there is no enforcement requiring the user to actually open a position (add liquidity) in the active bin. The fees are added, but the `cert.total_amount_a` and `cert.total_amount_b` do not include these amounts.
3.If the user opens a position with very large values (only by passing parameters, without actually adding liquidity), the bin's fee growth global will be inflated.
4.The first user can then exploit the difference between the position's fee growth entry and the inflated global value to drain funds from the pool.
The same issue also exists in `new_add_liquidity_cert`.

Suggestion:

Do not place validations only in the router contract; all critical validations must also be enforced within the inner functions that users can directly call.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-2 Incorrect Permission Check in `collect_position_reward`

Severity: Major

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/pool.move#630

Descriptions:

The `collect_position_reward` function uses the wrong permission check:

```
pool.operation_check(
    restriction::collect_fee_kind(),  // Should be collect_reward_kind()
    ctx.sender(),
    some(object::id(position)),
    config,
);
```

The function validates against an incorrect operation type, causing the access control system to evaluate the wrong permission set instead of the intended reward collection permissions.

Suggestion:

Replace with correct permission check: `collect_reward_kind()`

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-3 Missing Position-Pool Matching Validation in `add_liquidity` and `remove_liquidity` Functions

Severity: Medium

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/pool.move#396,489

Descriptions:

The `add_liquidity` and `remove_liquidity` functions in sources/pool.move lack validation to ensure that the provided position belongs to the specified pool.

Suggestion:

Add position-pool matching validation to both functions.

```
assert!(object::id(pool) == position.pool_id(), EPositionPoolNotMatch);
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-4 Ref Fee Calculation Based on Protocol Fee Leads to Double Charging

Severity: Medium

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/pool.move#1905-1915

Descriptions:

There is a significant fee double-counting issue in the fee calculation logic of the swap operation. It specifically manifests in the following code snippet:

```
swap_result.ref_fee = calculate_fee_inclusive(protocol_fee, ref_fee_rate);
swap_result.protocol_fee = protocol_fee;
if (a2b) {
    pool.protocol_fee_a = pool.protocol_fee_a + protocol_fee;
} else {
    pool.protocol_fee_b = pool.protocol_fee_b + protocol_fee;
};
```

The problem lies in: `protocol_fee` is calculated from the fee of each swap step. `ref_fee` is then calculated based on the entire `protocol_fee`, meaning the `ref_fee` is extracted from the protocol fee. However, the code accumulates the full `protocol_fee` into the pool's protocol fee without subtracting the portion already allocated to the referrer. This results in the protocol fee containing the part that should have been allocated to the referrer, causing fee double-counting.

Suggestion:

The code logic should be modified to ensure the protocol fee correctly deducts the referral fee portion:

```
swap_result.ref_fee = calculate_fee_inclusive(protocol_fee, ref_fee_rate);
swap_result.protocol_fee = protocol_fee - swap_result.ref_fee;  // Subtract referral fee from protocol fee
if (a2b) {
    pool.protocol_fee_a = pool.protocol_fee_a + swap_result.protocol_fee;  // Use adjusted protocol fee
} else {
    pool.protocol_fee_b = pool.protocol_fee_b + swap_result.protocol_fee;  // Use adjusted protocol fee
};
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-5 Dead Position Risk

Severity: Medium

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/pool.move#1033,1591

Descriptions:

1. `new_open_position_cert` logic

```
public fun new_open_position_cert<CoinTypeA, CoinTypeB>(
    pool: &mut Pool<CoinTypeA, CoinTypeB>,
    lower_bin_id: u32,
    width: u16,
    ...
): (Position, OpenPositionCert<CoinTypeA, CoinTypeB>) {
    ...
    let lower_bin_id = i32::from_u32(lower_bin_id);
    let upper_bin_id = lower_bin_id.add(i32::from(width as u32)).sub(i32::from(1));
    let mut position = pool
        .position_manager
        .open_position<CoinTypeA, CoinTypeB>(
            pool_id,
            pool.index,
            lower_bin_id,
            upper_bin_id,
            pool.url,
            ctx,
        );
    ...
}
```

If `width == 0`, then `upper_bin_id = lower_bin_id + 0 - 1 = lower_bin_id - 1`, which results in `lower_bin_id > upper_bin_id`. This creates an invalid position range, but the code does not

strictly enforce `width > 0`.

2. Subsequent liquidity removal

```
public fun remove_liquidity<CoinTypeA, CoinTypeB>(
    pool: &mut Pool<CoinTypeA, CoinTypeB>,
    position: &mut Position,
    bins: vector<u32>,
    liquidity_shares: vector<u128>,
    ...
) {
    ...
    let min_bin_id = i32::from_u32(bins[0]);
    let max_bin_id = i32::from_u32(bins[bins.length() - 1]);
    assert!(
        min_bin_id.gte(position.lower_bin_id()) && max_bin_id.lte(position.upper_bin_id()),
        EInvalidBinRange,
    );
    ...
}
```

If `position.lower_bin_id > position.upper_bin_id`, then no valid bins can satisfy the assertion, meaning liquidity removal is impossible.

3. Position lock The user's position can never be removed, causing funds to be locked permanently.

Impact

When `width == 0` in `new_open_position_cert`, a position with `lower_bin_id > upper_bin_id` is created.

Later, in functions like `remove_liquidity`, the check `min_bin_id.gte(position.lower_bin_id()) && max_bin_id.lte(position.upper_bin_id())` always fails.

This leads to irreversible locked positions, permanently trapping user funds.

Suggestion:

Enforce validation at position creation:

Ensure `width > 0`.

Ensure that width matches the actual number of bins being added. This prevents the creation of invalid positions and eliminates the dead-position risk.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-6 Position Boundary Check Bypass Vulnerability

Severity: Medium

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/pool.move#1077

Descriptions:

First, look at the `new_open_position_cert` function: Here, the position's `lower_bin_id` and `upper_bin_id` are indeed set correctly.

Then look at the `open_position_on_bin` function: Here, it only checks whether `bin_id` is equal to `next_bin_id`, but does not check whether `bin_id` is within the range of position's `lower_bin_id` and `upper_bin_id`.

The `repay_open_position` function also only checks whether `width == position.width()`. `position.width()` is `position.upper_bin_id.sub(position.lower_bin_id).add(i32::from(1))`, so as long as you continue to add positions in the `open_position_on_bin` function, you can keep adding more positions.

However, when removing liquidity, these positions can never be removed because of the check.

Suggestion:

Add the following assertion at an appropriate point in the `open_position_on_bin` function (for example, immediately after obtaining the `bin_id`):

```
assert!(
bin_id.gte(cert.position_info.lower_bin_id()) &&
bin_id.lte(cert.position_info.upper_bin_id()),
EBinIdOutOfPositionRange
);
```

This ensures that liquidity is added only to bins within the certificate range, preventing subsequent withdrawal failures.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-7 Users are Not Paying Protocol Fees

Severity: Medium

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/pool.move#306-392

Descriptions:

In the `open_position` and `add_liquidity` function, the fee calculation is as follows:

```
if (bin_id.eq(pool.active_id)) {
    (fee_a, fee_b) = apply_active_bin_composition_fees(pool, amount_a, amount_b, clk);
    amount_a = amount_a - fee_a;
    amount_b = amount_b - fee_b;
};
total_amount_a = total_amount_a + amount_a;
total_amount_b = total_amount_b + amount_b;
```

The total amounts (total_amount_a, total_amount_b) recorded in the final receipt are calculated after subtracting the fees (fee_a, fee_b):

```
AddLiquidityReceipt {
    pool_id: object::id(pool),
    amount_a: total_amount_a,
    amount_b: total_amount_b,
}
```

This correctly reflects the net amount of assets the user adds to the pool after paying the fees. This only records the increase in fees, without the actual transfer process.

Suggestion:

Consider modifying the AddLiquidityReceipt to explicitly include the fees paid by the user, or ensure the fee calculation and state update clearly reflect that the fees are a portion of the

user's total fee payment, not an additional amount created out of thin air. The current implementation might obscure the fact that the user is funding the fees through their deposited assets.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-8 Missing Validation for Length Consistency of `bins` `amounts_a` and `amounts_b` in `add_liquidity`

Severity: Minor

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/pool.move#771-822

Descriptions:

In the `pool.move` file, the `add_liquidity` function takes three vector parameters: `bins`, `amounts_a`, and `amounts_b`, and uses them within a loop:

```
while (idx < bins.length()) {
    let bin_id = i32::from_u32(bins[idx]);
    let (mut amount_a, mut amount_b) = (amounts_a[idx], amounts_b[idx]);
    // ...
    idx = idx + 1;
};
```

However, unlike the `open_position` function, the `add_liquidity` function does not validate whether the lengths of these three vectors are consistent. The open_position function includes an explicit check:

```
assert!(
    length == amounts_a.length() && length == amounts_b.length(),
    EInvalidAmountsOrBinsLength,
);
```

If `amounts_a` or `amounts_b` is longer than bins, the extra elements will be ignored. This behavior may be unintended and could lead to fund loss or unexpected liquidity distribution.

Suggestion:

Add the same length validation in the `add_liquidity` function as in `open_position`.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-9 Event Log and State Inconsistency May Mislead Users

Severity: Minor

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/pool.move#2827

Descriptions:

In the `add_reward` function, two independent parameters are used to represent the reward amount:

`reward_coin: Coin<CoinTypeC>`

`amount: u64`

However, the actual amount of tokens added to the pool's reward manager comes from the `reward_coin` object, while the `AddRewardEvent` event records the reward amount based on the `amount` parameter.

The code does not check whether these two values are equal. A malicious user could pass in a very small-value `reward_coin` while providing a very large `amount` parameter. This would cause the on-chain event to log a grossly inflated reward amount, while the pool itself only received a negligible reward.

Suggestion:

Remove the `amount`: u64 parameter from the `add_reward` function.

Inside the function, directly obtain the actual token amount from the `reward_coin` object, and use this value both for updating the pool state and for the `AddRewardEvent`.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POS-1 Typographical Error in Variable Name in `init` Function

Severity: Informational

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/position.move#212

Descriptions:

In the module's `init` function, the local variable `vaules` used for initializing the display object is misspelled. The correct spelling should be `values`. Although this typo does not affect the functional logic of the contract, it is a code quality issue that should be corrected.

Suggestion:

Correct the variable name from `vaules` to `values`.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POS-2 Misleading Error Code Used in `restore_position_info` Function

Severity: Informational

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/position.move#277

Descriptions:

The function `restore_position_info` is responsible for adding a `PositionInfo` back into the `PositionManager`. It contains the assertion:

`assert!(!manager.positions.contains(id), EPositionNotFound);`

This is intended to ensure that a position with the same ID does not already exist. However, if the assertion fails (the position already exists), the transaction aborts with the error code `EPositionNotFound`. The literal meaning of this error code is “position not found,” which is the opposite of the actual situation (the position was found). This can cause significant confusion for developers and auditors.

Suggestion:

To improve code readability and maintainability, define and use a more semantically accurate error code, such as `EPositionAlreadyExists`. This will more clearly convey the reason for the assertion failure.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

REG-1 Missing Token Pair Sorting in Pool Key Generation

Severity: Minor

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/registry.move#180

Descriptions:

```
public fun new_pool_key<CoinTypeA, CoinTypeB>(bin_step: u16, base_factor: u16): ID {
    let mut vec_bytes = type_name::get<CoinTypeA>().into_string().into_bytes();
    vec_bytes.append(type_name::get<CoinTypeB>().into_string().into_bytes());
    vec_bytes.append(bcs::to_bytes(&bin_step));
    vec_bytes.append(bcs::to_bytes(&base_factor));
    object::id_from_bytes(hash::blake2b256(&vec_bytes))
}
```

The `new_pool_key` function does not sort CoinTypeA and CoinTypeB before generating the pool key, which can lead to the creation of duplicate pools for the same token pair in different orders.

Suggestion:

Implement token pair sorting to ensure consistent pool key generation.

Resolution:

No longer restrict the order of token pairs.

REG-2 Ambiguous Boundary Definition for active_id

Severity: Minor

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/registry.move#126

Descriptions:

The active_id boundary check in `create_pool_internal` lacks clear design specification:

```
assert!(active_id.gt(min_bin_id()) && active_id.lt(max_bin_id()), EInvalidActiveId);
```

Uses strict inequalities (gt/lt) excluding boundary values ±443636, but design intent is unclear - should boundaries be valid or invalid?

Suggestion:

Clarify boundary design specification, if inclusive, use gte/lte operators.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

REG-3 Missing Version Check

Severity: Informational

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/registry.move#103-111

Descriptions:

The function `destory_receipt` lacks a version check mechanism.

```
#[allow(lint(share_owned))]
public fun destory_receipt<CoinTypeA, CoinTypeB>(
    receipt: CreatePoolReceipt,
    pool: Pool<CoinTypeA, CoinTypeB>,
) {
    let CreatePoolReceipt { pool_id } = receipt;
    assert!(pool_id == object::id(&pool), ECreatePoolCertNotMatch);
    transfer::public_share_object(pool);
}
```

This function does not perform any version validation.

Suggestion:

It is recommended to add a version check.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

REW-1 Missing Reward Release Update When Breaking from Settlement Loop

Severity: Medium

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/reward.move#190

Descriptions:

```
        while (op_next_settle_at.is_some() &&  current_time > settled_at) {
            let next_settle_at = option_u64::borrow(&op_next_settle_at);
            let time_delta = current_time.min(next_settle_at) - settled_at;
            accumulate_reward =
                accumulate_reward + (time_delta as u128) * reward.current_emission_rate;
            if (current_time < next_settle_at) {
                break
            };
            let emission_rate_net = *reward.period_emission_rates.borrow(next_settle_at);
            let next_emission_rate = i128::from(reward.current_emission_rate).add(
                emission_rate_net,
            );
            assert!(!next_emission_rate.is_neg(), ERewardRateUnderflow);
            reward.current_emission_rate = next_emission_rate.as_u128();
            reward.reward_released = reward.reward_released + (accumulate_reward as
u256);
            settled_at = next_settle_at;
            let node = skip_list::borrow_node(&reward.period_emission_rates, next_settle_at);
            op_next_settle_at = skip_list::next_score(node);
            if (next_settle_at < current_time) {
                reward.period_emission_rates.remove(next_settle_at);
            };
        };
```

In the `settle_rewards` function, when the condition `if (current_time < next_settle_at)` is met, the function breaks from the inner loop without updating `reward.reward_released` with the accumulated reward. This leads to potential reward loss and state inconsistency.

Suggestion:

Add reward release update before breaking from the loop.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

REW-2 Accumulated but Unwithdrawable Refunded Rewards in `reward_refunded`

Severity: Medium

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/reward.move#329-336

Descriptions:

In the Cetus DLMM protocol, when liquidity is zero, rewards are refunded and accumulated in the `reward_refunded` field. However, the code provides no mechanism to withdraw or utilize these refunded rewards. Specifically: The `refund_rewards` function accumulates unallocated rewards into the `reward_refunded` field:

```
public(package) fun refund_rewards(manager: &mut RewardManager,
accumulated_rewards: vector<u128>) {
    let mut idx = 0;
    while (idx < manager.rewards.length()) {
        let reward = manager.borrow_mut_reward(idx);
        reward.reward_refunded = reward.reward_refunded +
        (accumulated_rewards[idx] as u256);
        idx = idx + 1;
    };
}
```

Although there is a getter function for `reward_refunded`, there is no corresponding withdrawal function.

Suggestion:

Implement an admin function to allow the protocol owner to withdraw the refunded rewards.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

REW-3 Inflating `reward_released` Statistics

Severity: Minor

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/reward.move#292

Descriptions:

There is an accounting logic error in the inner loop of reward settlement that causes the total released rewards to be repeatedly accumulated:

```
let mut accumulate_reward = 0;
let mut settled_at = last_updated_time;
while (op_next_settle_at.is_some() && current_time > settled_at) {
    let next_settle_at = option_u64::borrow(&op_next_settle_at);
    let time_delta = current_time.min(next_settle_at) - settled_at;
    accumulate_reward = accumulate_reward + (time_delta as u128) *
reward.current_emission_rate;

    // ... update emission rate ...
    reward.current_emission_rate = next_emission_rate.as_u128();
    reward.reward_released = reward.reward_released + (accumulate_reward as u256); //
Error: using cumulative total
    settled_at = next_settle_at;
    // ... continue loop ...
};
```

Specific Issues:

1. The `accumulate_reward` variable continuously accumulates the total rewards across all time segments in the loop
2. At each emission rate change node, the code adds the entire `accumulate_reward` (cumulative total from start to current) to `reward_released`

3. When multiple rate change nodes exist, rewards from earlier time periods are repeatedly counted by subsequent nodes

Impact:

- Causes `reward_released` statistics to be systematically inflated, potentially several times the actual value
- Affects the accuracy and transparency of the protocol's reward accounting
- May cause anomalies in downstream functions that depend on these statistics (such as reward display, auditing, etc.)
- In extreme cases, may affect the correctness of reward distribution

Suggestion:

Accumulate by Segment Increment

```
let mut accumulate_reward = 0;
let mut settled_at = last_updated_time;
while (op_next_settle_at.is_some() && current_time > settled_at) {
    // ...

    let segment_reward = (time_delta as u128) * reward.current_emission_rate;
    accumulate_reward = accumulate_reward + segment_reward;

    // ... update emission rate ...
    reward.current_emission_rate = next_emission_rate.as_u128();
    reward.reward_released = reward.reward_released + (segment_reward as u256); // Fix:
only accumulate this segment's increment
    settled_at = next_settle_at;
    // ... continue loop ...
};
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

PAR1-1 Missing Validation of Current Time vs End Time in `create_partner`

Severity: Minor

Status: Fixed

Code Location:

packages/cetus_dlmm/sources/partner.move#224-275

Descriptions:

An incomplete time validation logic issue has been identified in the `create_partner` function within `partner.move`. Missing Validation: There is no check to ensure that `end_time` is greater than the current time. This allows the creation of a partner that has already expired at the moment of creation.

`

Suggestion:

Check that the end time is greater than the current time.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

